`Card::getFullName()` is now accessing the `value` property of the `Suit` case, from which we obtain its scalar value:

```
$card = new Card(Suit::SPADES, 'ace');
print "The {$card->getFullName()}, the {$card->getFullName()} \m/" .
PHP_EOL;
```

```
Output:
The ace of spades, the ace of spades \m/
```

Backed enums are somewhat restricted. The entire enum must be declared either `int` or `string` (and it's not possible to use a union of `int|string`), and every case must have a unique value.

## Enums as Classes

So far, enums look a bit like classes, and enum values look like static properties. We even get OOP-flavored errors when we misuse enums. In fact, enums *are* classes, or rather, they are built on classes. PHP implements enum cases as (single) object instances of the enum "class":

```
var_dump(Suit::class);
var_dump(gettype(Suit::CLUBS));
var_dump(get_class(Suit::CLUBS));
var_dump(Suit::CLUBS->value);
```

```
Output:
string(4) "Suit"
string(6) "object"
string(4) "Suit"
string(5) "clubs"
```

It is even possible to do class-like things with enums. For example, we can implement an interface, then use that interface in a type declaration:

```
interface CastSpell
{
    public function cast(): string;
}
```